

Word-Group Guided Multi-Token Prediction with Adaptive Loss Weighting for Hindi and Marathi

What's done, what's next, who builds what, and the exact interfaces that let each piece be written independently and plug together.

Jainam

Modeling & Training

Jai

Data & Linguistics

Om

Evaluation & Infrastructure

Companion files in the repo under `docs/plan/` : `README.md` · `INTERFACES.md` · `tasks/JAINAM.md` · `tasks/JAI.md` · `tasks/OM.md` . Each task file is written to be pasted into Claude together with `INTERFACES.md` to generate the code for one task at a time.

1 • Where we are

3.05

NTP baseline held-out loss (100k sentences)

8.10 → 5.84

MTP k=2, head 1 held-out loss over 12.5k batches

1.33

words per group on real corpus text (rules v0)

25%

words attached to a group by rules v0

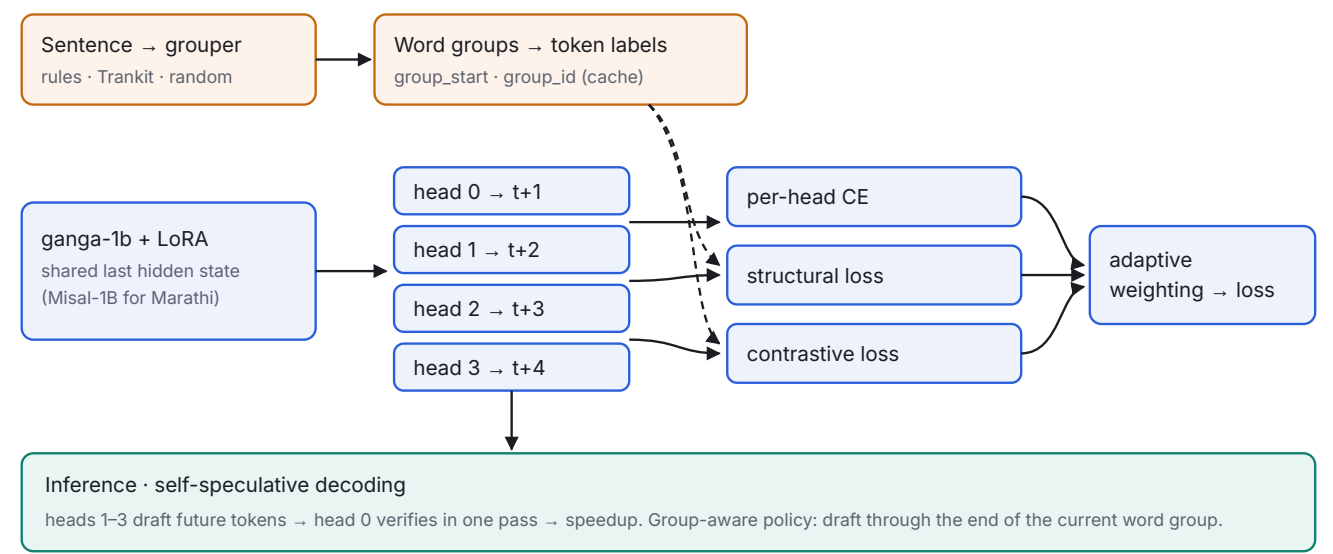
The pipeline already works end to end on one laptop GPU (RTX, 8 GB).

Piece	Result
Base model	LingoIITGN/ganga-1b (1B, Hindi) loads and generates. Tokenizer gives clean Devanagari subwords
LoRA	r=8 on q_proj, v_proj, lr 2e-4, grad clip 1.0, batch 8, max_len 128
Data	ai4bharat/IndicCorpV2 → indiccorp_v2 / hin_Deva, streamed
NTP baseline	100k sentences. Held-out loss 3.05 . Flat across lr/rank sweeps: the base model is already well fit to this distribution, and that is a finding we report
Token-level MTP baseline (k=2, linear head)	12,500 batches. Head 0 3.09 , head 1 8.10 → 5.84
Word groups v0	Rule-based (auxiliary + postposition lists). On real corpus text: 25% of words attach, 1.33 words/group
Token alignment	Word groups → token labels via offset_mapping, verified
Medusa wrapper	MedusaWrapper with linear extra heads, bf16-safe

What's missing: the actual contribution (structural + contrastive losses, adaptive weighting), Marathi, a proper evaluation suite (FLORES, per-head accuracy, speculative-decoding speedup), and a reproducible setup on Kaggle.

2 • The idea

Hindi and Marathi pack one meaning unit into several tokens: **जा रहा था** (was going), **घर से** (from home), **के बारे में** (about). If the far-ahead prediction heads are told where those units start and end, they should learn faster, be more accurate inside a unit, and give longer accepted drafts at inference time.



Colour = owner. **Jai** data and grouping · **Jainam** model, losses, training · **Om** inference and evaluation.

3 • Roles

Person	Owns
Jainam: Modeling & Training	Model heads, all loss functions, adaptive weighting, <code>train.py</code> , every training run, group-aware decoding policy, method + results analysis
Jai: Data & Linguistics	Word grouping (Hindi + Marathi, rules + Trankit + random control), gold annotation set, grouper scoring, boundary-label cache on Kaggle, group statistics + error analysis
Om: Evaluation & Infrastructure	Package structure, config/logging/checkpoints, Kaggle notebooks, evaluation suite (perplexity, per-head accuracy, speculative decoding), all eval runs, tables + figures

All training runs happen in one place (Jainam's Kaggle account), so checkpoints, seeds and logs stay consistent. Evaluation runs on Om's and Jai's Kaggle accounts, which spreads the GPU quota across three accounts.

4 • Order of work

No dates, just order. Phases A–C are the current priority. Experiments and write-up come later. Inside a phase the three tracks run in parallel; inside a cell, go top to bottom. **H1–H7** mark handoffs (next page).

Phase A • Foundation start immediately • priority

JAINAM

JN-1 Heads module: k heads, `linear` + zero-init `resblock`, per-position CE

JN-2 Structural-loss design note (S1 / S2 / S3)

JN-3 `scripts/train.py` driven by YAML (stub Om's parts until H1)

JAI

JI-1 Hindi rule grouper v1: full lists, compound postpositions, light verbs

JI-6a Hindi boundary cache, rules grouper → **H3**

JI-2 Annotation guide + 200-sentence Hindi gold set

OM

OM-1 Package skeleton, `legacy/`, config, dtype, logging, collate → **H1**

OM-2 Checkpoint save / load / resume

OM-3 Kaggle training notebook with resume → **H2**



Phase B • Baselines on Kaggle + grouping quality needs H2 • priority

JAINAM

JN-4 Reproduce R0, R1 on Kaggle; train R2 (k=4 `resblock`)

JAI

JI-3 Trankit grouper

JI-4 Random grouper (control)

JI-5 Score groupers vs gold, pick default → **H5**

JI-6b Add trankit + random caches

OM

OM-4 Eval: per-head ppl + top-k, in-group vs boundary → **H4**

OM-5 Blind double-annotation (50)

Gate: Kaggle R0 / R1 within ~0.05 of the laptop numbers before moving on.



Phase C • The method needs H3, H4 • priority

JAINAM

JN-5 Structural loss (pilots → pick)

JN-6 Contrastive loss

JN-7 Adaptive weighting: fixed / uncertainty / DWA

JAI

JI-7 Marathi rules + 150 gold + Marathi cache → **H7**

JI-8 Group statistics: same-group rate per lookahead

OM

OM-6 Self-speculative decoding engine + baseline policies → **H6**

OM-7 `evaluate.py` + Kaggle eval notebook

LATER • NOT A PRIORITY YET

Phase D + E • Later: experiments and write-up not a priority yet • start after C

JAINAM

JN-8 Run matrix R3 → R10

JN-9 Group-aware draft policy (needs H6)

§ Method, results, conclusion

JAI

JI-9 Error analysis by group type

§ Data + annotation, related work

OM

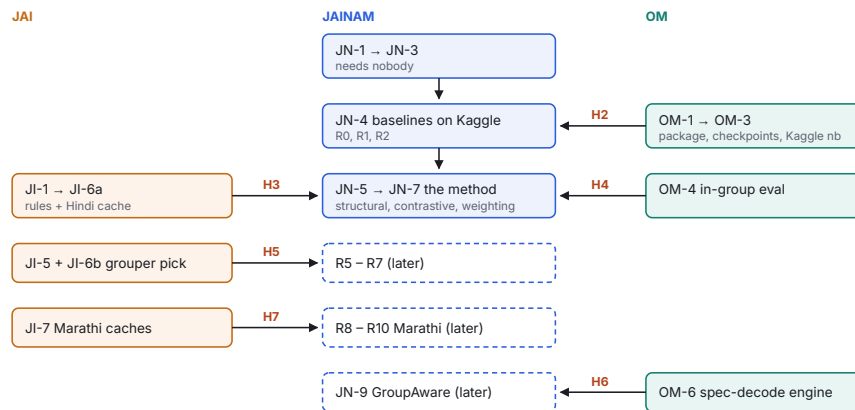
OM-8 Evaluate every run

OM-9 Tables + figures, incl. lookahead heatmap

§ Setup, evaluation, figures

5 • Dependencies and handoffs

Most of the work is independent. These are the only points where one person has to finish something before someone else can move. Each handoff = a merged PR or published Kaggle Dataset + a message in the team chat.



Critical path: H2 and H3 are what can block training. So Om's first three tasks (OM-1 → OM-3) and Jai's first two (JI-1 → JI-6a) come before everything else on their lists. The gold set, Trankit and the random control matter for the final results but not for getting the method running, so they come after.

Nobody waits on Jainam until H10. Jainam's early tasks (JN-1 → JN-3) don't need anyone else's work, because they only use the contracts.

#	Handoff	From → To	What it unblocks	Until it lands
H1	Package skeleton + config / device / collate / logging merged (OM-1)	Om → everyone	Everyone writes code inside mtp/	Write your module in a standalone file using the signatures in INTERFACES.md, then move it in
H2	Checkpointing + Kaggle training notebook (OM-2, OM-3)	Om → Jainam	Running anything on Kaggle (JN-4 onwards)	JN-1 → JN-3, testing on the laptop
H3	Hindi boundary cache, rules grouper (JI-1 → JI-6a)	Jai → Jainam, Om	Training with structural / contrastive losses (JN-5, JN-6); in-group eval split (OM-4)	Unit-test the losses on hand-built batches; run baselines R0-R2 (they need no labels)
H4	Eval with the in-group / boundary split (OM-4)	Om → Jainam	Choosing the structural-loss variant from pilot runs (JN-5)	Pilots log their own eval losses; pick once H4 lands
H5	Grouper decision + trankit / random caches (JI-5, JI-6b)	Jai → Jainam	Final runs R5, R6, R7	Develop the method on the rules cache from H3
H6	Speculative-decoding engine (OM-6)	Om → Jainam	GroupAware draft policy (JN-9)	— (JN-9 is in the later block anyway)
H7	Marathi caches (JI-7)	Jai → Jainam	Marathi runs R8-R10	Hindi runs
H8	50 blind annotations (OM-5)	Om → Jai	Agreement number in JI-5	Score groupers without κ, add it when it arrives
H9	Gold sentence IDs (JI-2)	Jai → Om	OM-5	—
H10	Trained runs (JN-4, JN-8) published as Kaggle Datasets	Jainam → Om	Evaluating runs (OM-8)	Test eval on R1/R2 checkpoints

6 • Run matrix

Same seed, same eval sets, same step budget unless noted. Every run gets a YAML in `configs/`.

ID	Lang	Heads	Head type	Extra losses	Weighting	Grouper	Why
R0	hi	1	–	–	–	–	NTP baseline
R1	hi	2	linear	–	fixed	–	Reproduce laptop MTP
R2	hi	4	resblock	–	fixed	–	Token-level MTP baseline (the real comparison)
R3	hi	4	resblock	structural	fixed	best	Method, part 1
R4	hi	4	resblock	structural + contrastive	fixed	best	Method, part 2
R5	hi	4	resblock	structural + contrastive	adaptive	best	Full method
R6	hi	4	resblock	structural + contrastive	adaptive	random	Control: does linguistic structure matter?
R7	hi	4	resblock	structural + contrastive	adaptive	other grouper	Grouper ablation
R8	mr	1	–	–	–	–	Marathi NTP
R9	mr	4	resblock	–	fixed	–	Marathi MTP
R10	mr	4	resblock	structural + contrastive	adaptive	mr best	Marathi full method

Optional if compute allows: $k \in \{2, 3, 4, 6\}$ sweep on R5; a second seed for R2 and R5.

Marathi base model: `smallstepai/MisaL-1B-instruct-v0.1` (Llama architecture). Fallback if it fails to load: keep `ganga-1b` and report Marathi as cross-lingual transfer.

7 • Ideas that make this stand out

- 1 Random-boundary control (R6).** Same losses, but boundaries drawn at random with the same group-length distribution. If R5 beats R6, the gain comes from linguistic structure, not just from having an extra loss. This is the strongest single experiment in the paper.
- 2 In-group vs at-boundary accuracy.** Split every head's accuracy by whether the target token continues a group or starts a new one. We expect the structural loss to raise in-group accuracy of the far heads, and this shows where it helps.
- 3 Group-aware speculative decoding.** Instead of always drafting $k-1$ tokens, draft up to the predicted end of the current word group. Report mean accepted length and wall-clock speedup against `FixedK` and `ConfidenceCut`.
- 4 Group Integrity score.** Fraction of accepted draft spans that end on a group boundary rather than mid-group. A new, simple metric tied to our hypothesis.
- 5 Lookahead heatmap figure.** One sentence, tokens coloured by which head first predicts them correctly, with word-group brackets drawn above. This is the figure that explains the paper at a glance.
- 6 Hindi vs Marathi contrast.** Hindi is analytic (separate postpositions), Marathi is agglutinative (suffixes inside the word). Groups should be fewer words but more tokens in Marathi. Report how the method behaves across that difference.

8 • Working rules

- **Branches:** `jainam/*`, `jai/*`, `om/*`. Merge to `main` through a PR that the other two can see. Keep PRs small.
- **Contracts first:** never change a signature in `INTERFACES.md` silently. Edit the file in the same PR and say so in the PR title.
- **Tests:** every module gets a CPU-only pytest with a tiny input (2–3 sentences, a mocked tiny model where needed). `pytest -q` must pass before merge.
- **No big files in git:** checkpoints, caches and logs live on Kaggle (`/kaggle/working` , Kaggle Datasets). `results/*.json` and figures are committed.
- **Windows:** set `$env:PYTHONUTF8 = "1"` before running anything that reads Devanagari text.
- **Kaggle GPUs (T4 / P100)** don't have native bf16. `mtp/device.py` picks fp32 weights + fp16 autocast + GradScaler there, and bf16 where supported. Never hard-code `torch.bfloat16`.
- **Using Claude to write code:** paste `INTERFACES.md` + your task file + the specific task ID. Ask for the module plus its test. Run the test before pushing.

9 • Using Claude to generate the code

1. Pull `main` , create your branch (`jai/JI-3-trankit`).
2. Open a new Claude chat. Paste the **Context** block of your task file, then all of `INTERFACES.md` , then: *"Do JI-3. Write the module and a CPU-only pytest. Match the contracts exactly."*
3. Run the test locally. Fix until green. Run the module once on real data.
4. PR to `main` with what it does, test output, and any number it produced.

Jainam: Modeling & Training

How to use this file with Claude: paste the "Context" block, then `INTERFACES.md`, then the task ID. For

`JN-2` / `JN-5` / `JN-6` / `JN-7`, have Claude argue the design with you before writing code. Branch:

`jainam/<task>`.

Context (paste this to Claude)

We are building *Word-Group Guided Multi-Token Prediction with Adaptive Loss Weighting for Hindi and Marathi*. Base: LingoIIT6N/ganga-1b (1B, SentencePiece, vocab 30k), LoRA $r=8$ on `q_proj` / `v_proj`, $lr\ 2e-4$, grad clip 1.0, batch 8, max_len 128, IndicCorpV2 `hin_Deva`.

Results so far (laptop, 8 GB GPU):

- NTP baseline: held-out loss 3.05, flat. A sweep showed $lr\ 2e-3$ with a larger LoRA diverges, and $lr\ 5e-4$ with $r=32$ slowly degrades. The base model is already well fit to this distribution.
- Token-level MTP, $k=2$, `MedusaWrapper` with a **randomly initialised** `nn.Linear(hidden, vocab)` extra head on the last hidden state, losses summed: 12,500 batches, head 0 $3.06 \rightarrow 3.09$, head 1 $8.10 \rightarrow 5.84$.
- Word groups: rule-based grouper (aux + postposition lists), aligned to tokens via `offset_mapping` $\rightarrow$ per-token `group_start` and `group_id`.

Code lives in the `mtp/` package (`INTERFACES.md`). Jai provides groupers + a pre-labelled boundary cache. Om provides config, device/dtype handling, logging, checkpointing, Kaggle notebooks and the evaluation suite including the speculative-decoding engine. I own the model, every loss, the weighting scheme, `scripts/train.py`, all training runs, and the group-aware draft policy.

Waiting points: `JN-1` $\rightarrow$ `JN-3` need nobody (code against `INTERFACES.md`, stub what isn't merged). `JN-4` needs H2 (Om: checkpoint + Kaggle notebook). `JN-5` / `JN-6` real training needs H3 (Jai: Hindi cache), and the variant choice needs H4 (Om: in-group eval). R5–R7 need H5. Marathi needs H7. `JN-9` needs H6.

`JN-1` · Heads module

Files: `mtp/model/heads.py`, `mtp/losses/mtp_ce.py`

- Port `MedusaWrapper` $\rightarrow$ `MTPModel` (`INTERFACES` §5): `num_heads` counts head 0. Return `MTPOutput(logits, hidden, aux)`.
- `head_type="linear"`: current behaviour (for reproducing R1).
- `head_type="resblock"` (Medusa-1): `h_d = h + SiLU(W_d h)`, `logits = lm_head(h_d)`. **Init `W_d` to zero** so every head starts as a copy of the LM head, and reuse the base model's (frozen) `lm_head`. Two reasons: (1) it removes the $30k \times \text{hidden}$ randomly-initialised matrix that made head 1 start at loss 8.1; (2) each extra head costs only hidden^2 params, which matters on a 16 GB Kaggle GPU with $k=4$. Option `n_layers` per head (default 1).
- `mtp_ce.py`: `per_head_ce(logits_list, input_ids, attention_mask, reduction="none")` returns per-position losses `[B, T-d]` for each head, not just the mean, so the structural loss can re-weight them. Port the masking from `compute_head_loss`.
- Sanity test: with `resblock` and zero init, head d 's logits equal head 0's at every position.

`JN-2` · Structural-loss design note

File: docs/design_structural_loss.md (1–2 pages, decide before coding [JN-5](#))

The proposal says the auxiliary loss should penalise heads that *cross word-group boundaries inconsistently*. Turn that into math. Notation: head d ($d = 0 \dots k-1$) at position t predicts the token at $u = t+d+1$. Candidate formulations to evaluate:

- **S1 · Boundary-aware re-weighting.** For head d at position t , let $\text{same} = [\text{group_id}[t] = \text{group_id}[u]]$. Weight the per-position CE by $1 + \alpha \cdot \text{same}$, or the opposite (down-weight cross-group targets as near-unpredictable noise). Cheap, but it only re-weights existing CE, so it can't add new signal.
- **S2 · Boundary-prediction auxiliary head.** Each extra head d gets a tiny linear probe on h_d predicting $\text{group_start}[u]$ (binary CE). This teaches each head *where the next unit starts*. Bonus: at inference this probe is exactly what the `GroupAware` draft policy needs ([JN-9](#)).
- **S3 · In-group self-consistency.** Head d at t and head $d-1$ at $t+1$ both predict the **same** token u . When $t+1$ and u are in the same group, the far head should agree with the nearer head that has seen one more token: $\text{KL}(\text{sg}[p_{\{d-1\}(\cdot|t+1)}] \parallel p_{\{d\}(\cdot|t)})$, masked to in-group pairs ($\text{sg} = \text{stop-gradient}$). This is the most literal reading of "inconsistently crossing boundaries" and the most novel, but also the most expensive (extra logits alignment, so use top-k-sparse KL to save memory).

Recommended path: pilot S2 and S3 (and S2+S3) on 2,000-step runs, pick the best by head-1..3 in-group top-1 on `indicorp_eval`, write down the choice and the pilot numbers in the note. Those pilot numbers also go into the paper's ablation.

[JN-3](#) · Unified training script

File: scripts/train.py

- `python scripts/train.py --config configs/R5.yaml [--resume auto] [--set optim.lr=...]`
- Builds: tokenizer → base model (`mtp.device.pick_dtype`) → LoRA → `MTPModel` → losses from config → weighting module → AdamW over LoRA + heads + probes + weighting params (the weighting params get their own param group, no weight decay).
- Data: boundary cache via `datasets.load_from_disk` + Om's `Collator`. NTP/MTP runs without group losses ignore the group columns.
- Loop: `autocast_ctx` + `GradScaler` from `mtp.device`, optional grad accumulation, clip 1.0, `MetricLogger` every 20 steps (each loss term, each weight), eval on `eval_small` every `eval_every`, `save_run` every `save_every`. On `--resume auto`: `latest_step` → restore model, optimizer, scaler, RNG, data position.
- Deterministic: seed everything, and record git commit hash in `config.yaml`.
- `configs/`: write `R0.yaml` ... `R10.yaml` and `pilot_S2.yaml`, `pilot_S3.yaml`, `pilot_S23.yaml`.

[JN-4](#) · Reproduce baselines on Kaggle

- R0 (NTP) and R1 (MTP $k=2$ linear) through `train.py` on Kaggle. **Gate:** within ~ 0.05 of the laptop numbers (3.05 / 3.09 / 5.84). If not, the difference is in dtype handling (fp16 vs bf16), data order or masking, so find it before moving on.
- R2 ($k=4$ resblock). Expect head 1 to start near head 0's loss thanks to the zero-init, not at 8.1. This is our real token-level baseline.
- Publish each finished run as a Kaggle Dataset `mtp-run-<ID>` and post the name for Om.

[JN-5](#) · Structural loss

File: mtp/losses/structural.py

- Implement the variant(s) chosen in [JN-2](#) behind one interface: `StructuralLoss(variant, **kw)` (`mtp_output, batch`) → `dict[str, Tensor]` returning named scalar terms (e.g.

`{"struct/boundary_bce/h1": ..., "struct/consistency/h2": ...}`), so the weighting module can treat each term separately.

- Handle `group_id == -1` (special/pad) and positions where `t+d` runs off the sequence.
- Unit test: hand-built 1-sentence batch where you can compute the expected loss by hand.
- Pilot runs → pick → R3.

JN-6 • Contrastive loss

File: `mtp/losses/contrastive.py`

- Supervised contrastive (SupCon) over tokens: project `hidden` with a 2-layer MLP to 128-d, L2-normalise. Positives = other tokens with the same `(sentence, group_id)`, negatives = tokens from other groups in the batch. Temperature from config (start 0.1). Subsample ≤ 512 tokens per batch to bound memory.
- Variant to try if SupCon is flat: group-level InfoNCE, with mean-pooled group embedding vs the head-d hidden state `h_d` at the token just before the group.
- Watch for it hurting head 0 perplexity, since pulling token representations together can blur next-token information. Log head-0 loss vs R2 closely. If it hurts, apply the loss on the projection only (already the case) and lower the weight.
- When stable, this becomes run R4.

JN-7 • Adaptive loss weighting

File: `mtp/losses/weighting.py`

- `LossWeighting(scheme, term_names)` with `forward(dict_of_losses) → (total, dict_of_effective_weights)`.
- `fixed`: per-head weights 0.8^d (Medusa) for CE terms, and config weights for aux terms.
- `uncertainty` (Kendall et al. 2018): learnable $s_i = \log \sigma_i^2$, $total = \sum \frac{1}{2} \cdot \exp(-s_i) \cdot L_i + \frac{1}{2} \cdot s_i$. Clamp $s_i \in [-4, 4]$. Head 0's weight can be fixed at 1 so the LM doesn't get de-prioritised; make that a flag and test both.
- `dwa` (Dynamic Weight Average): $w_i(t) \propto \exp(r_i(t-1)/T)$, where $r_i = L_i(t-1)/L_i(t-2)$ over epoch-like windows of N steps.
- Log effective weights every 20 steps (they become Om's figure (c)).
- Pilot all three on top of R4's losses (2,000 steps), pick one → R5. Keep the others for the ablation table.

JN-8 • Run matrix

- Execute R3 → R10 from `README.md §5` in that order (R5 before R6/R7 because R6/R7 copy R5's config). For R6, ask Jai for the `random` cache built from the default grouper's length histogram.
- Marathi (R8-R10): base `smallstepai/Misal-1B-instruct-v0.1`. Check its tokenizer, `lm_head` name and LoRA target module names before launching. Fallback: `ganga-1b`.
- Keep `docs/runs.md`: run ID, commit, config, Kaggle dataset name, final numbers, notes.

JN-9 • Group-aware draft policy

File: `GroupAware` in `mtp/eval/draft_policy.py` (engine and contract from Om, INTERFACES §11)

- Use the S2 boundary probes: propose drafts from heads $d = 1, 2, \dots$ while (a) head d 's max prob $\geq \tau$ and (b) the probe says its target token is **not** a new group start, plus include the token that closes the group. Intuition: draft the rest of the current word group in one go, then re-verify.

- Fallback when a run has no probe: use Jai's rule grouper on the decoded text so far to tell whether we're mid-group.
 - Tune τ on 50 held-out prompts, report on FLORES. Compare against `FixedK` and `ConfidenceCut` in Om's Table 4.
-

Write-up (Phase E)

- **Method:** model + heads, structural loss (final variant + the pilot comparison), contrastive loss, weighting, group-aware decoding.
- **Results discussion:** main table, R5 vs R6 (does linguistics matter), in-group vs boundary split, decoding speedups, Marathi vs Hindi, limitations (flat NTP baseline, single seed, rule-based groups).
- **Conclusion + future work.**

Jai: Data & Linguistics

How to use this file with Claude: paste the "Context" block below, then `INTERFACES.md`, then the one task you're on (e.g. "Do `JL-3`"). Ask for the module **and** its pytest. Run the test, then open a PR on a `jai/<task>` branch.

Context (paste this to Claude)

We are building *Word-Group Guided Multi-Token Prediction for Hindi and Marathi*. A 1B Hindi LM (LingoIITGN/ganga-1b, SentencePiece tokenizer, vocab 30k) is fine-tuned with LoRA and extra "Medusa-style" heads that predict tokens $t+2$, $t+3$, $t+4$. Our hypothesis is that these far-ahead heads do better if they know where **word groups** begin and end. A word group is a multi-word meaning unit: a verb + auxiliary chain (जा रहा था), a noun + postposition (घर से), a compound postposition (के बारे में), a light-verb construction (कर दिया). My job is to produce those word groups reliably, measure how good they are, and ship them as token-level labels the training code can read.

Existing code in the repo (being moved to `Legacy/`):

- `word_group_boundaries.py`: `find_word_groups(sentence) → list[list[str]]`. A word joins the previous group if (after stripping `।, . ! ?`) it is in `AUXILIARIES` or `POSTPOSITIONS`. Lists are short (21 auxiliaries, 8 postpositions).
- `boundary_alignment.py`: tokenizes the full sentence with `return_offsets_mapping=True` and marks a token as a group start if a group's start char falls in `[tok_start, tok_end)`, which handles SentencePiece's leading space. Special tokens get 0.
- `validate_on_real_data.py`: coverage on real IndicCorp text. Last result was 25.0% of words attaching and 1.33 words/group.

Dataset: `load_dataset("ai4bharat/IndicCorpV2", "indicorp_v2", split="hin_Deva", streaming=True)` (Marathi: `mar_Deva`). FLORES-200 devtest for clean evaluation text.

JL-1 · Hindi rule grouper v1

File: `mtp/data/grouping/hindi_rules.py` (class `HindiRuleGrouper`, `name="hi_rules_v1"`), `mtp/data/grouping/base.py` (Protocol + `get_grouper` registry)

Start from `find_word_groups` and extend it:

- Auxiliaries / aspect / modal:** है हैं था थी थे हो होगा होगी होंगे हूँ रहा रही रहे गया गई गए सकता सकती सकते चुका चुकी चुके पाया पाई पाए
- Light / vector verbs that attach to a preceding verb:** लगा लगी लगे पड़ा पड़ी पड़े दिया दी दिए लिया ली लिए डाला डाली डाले
- Single postpositions / particles:** का की के को से में पर ने तक भी ही लिए साथ बारे पास नहीं वाला वाली वाले
- Compound postpositions (multi-word windows, match longest first):** के लिए · के साथ · के बारे में · के बाद · के पास · के अंदर · के बाहर · के ऊपर · के नीचे · के दौरान · के अनुसार · की ओर · की तरह · के कारण
- Punctuation: strip `।।।, . ! ? ; : ' ' ( )` for lookup only, and keep the original word.
- Put the word lists in a module-level dict so they're easy to edit and can be cited in the report.

Checks (pytest):

- "मैं कल बाजार जा रहा था।" → `[["मैं"], ["कल"], ["बाजार"], ["जा", "रहा", "था"]]`
- "वह घर के बारे में बात कर रहा है" groups घर के बारे में and बात / कर रहा है correctly (write down what you decide and test it)

- Concatenated groups == `sentence.split()` for 1,000 random corpus sentences (property test)
- Re-run coverage stats and record the new % and words/group in the PR description

JI-2 • Annotation guideline + Hindi gold set

Files: `docs/annotation_guide.md`, `data/gold/hi_gold.jsonl` (format in INTERFACES §7)

- Write a 1–2 page guide: what counts as a group, with 3+ examples per type (aux chain, postposition, compound postposition, light verb, negation, `वाला`-forms) and what does **not** group (adjective + noun, coordinated words, numbers).
- Annotate **200 Hindi sentences**: 100 from FLORES-200 devtest `hin_Deva`, 100 from IndicCorp rows 1,000–5,000. Pick sentences of 6–25 words.
- Write a tiny helper `scripts/annotate.py` that shows a sentence with word indices and lets you type group spans (e.g. 3-5 7-8) and appends to the JSONL. This makes the annotation much faster.
- Hand 50 of them (IDs only, no groups) to Om for blind double-annotation (used in JI-5).

JI-3 • Trankit grouper

File: `mtp/data/grouping/trankit_grouper.py` (`name="trankit"`, supports `lang="hi"` and `"mr"`)

- Use Trankit's dependency parse. A word attaches to its head's group if its `deprel` is one of `aux`, `aux:pass`, `case`, `mark`, `compound:lv` (light verb), `cop`, or the word is `PART` with a negation feature. Then convert to contiguous groups (if an attachment is non-contiguous, keep the words separate).
- Map Trankit tokens back to whitespace words (Trankit may split punctuation). Must satisfy the `"concatenation == split()"` rule.
- Lazy-load the pipeline (no download at import). Add `batch_group_words(sentences)` for speed.
- If Trankit won't install on the Python version we use, try Stanza with the same `deprel` logic and name it `stanza`. Note which one worked in the PR.

JI-4 • Random grouper (control)

File: `mtp/data/grouping/random_grouper.py` (`name="random"`)

- Constructor takes a group-length distribution (a histogram dict `{1: 0.7, 2: 0.2, 3: 0.08, 4: 0.02}`) and a seed.
- `group_words` samples group lengths from that distribution until the sentence is covered, deterministic per (seed, sentence).
- Add `RandomGrouper.fit_from(grouper, sentences)` that measures the histogram from another grouper, so R6 matches the real grouper's length distribution exactly.

JI-5 • Score groupers against gold

Files: `mtp/eval/group_metrics.py`, `scripts/score_groupers.py`

- Boundary-level precision / recall / F1, where a "boundary" is a position between word `i` and `i+1` that is **inside** a group (i.e. an attachment). Also report exact-match group accuracy.
- Report per group type, using a type tag you add during annotation, or infer it from the attached word's list membership.
- Inter-annotator agreement on Om's 50: Cohen's κ over word-pair attach / don't-attach decisions.
- Output `results/grouping/hi_scores.json` and a markdown table. Compare `hi_rules_v1`, `trankit`, `random`.

- **Decision:** the grouper with the best F1 becomes the default for R3–R5 and the other one is R7. Post the decision in the team chat.

JI-6 • Boundary cache builder (Hindi)

Priority: this is what unblocks training (handoff H3). Do it **right after** JI-1, before the gold set: ship JI-6a = the hi_rules_v1 cache only (train / eval / flores), then later JI-6b = add trankit + random once JI-3 / JI-4 exist.

Files: mtp/data/boundary_cache.py, scripts/build_boundary_cache.py

- CLI: `python scripts/build_boundary_cache.py --lang hi --grouper hi_rules_v1 --split train --n 200000 --out boundary_cache/`
- Uses `label_tokens` from `align.py` (port of `boundary_alignment.py`, generalized to take any grouper and tokenizer). Train split = IndicCorp rows $\geq 1,000$; eval = rows 0–999; flores = FLORES devtest.
- Writes the HF dataset + `meta.json` (INTERFACES §3). Uses `datasets.map(batched=True, num_proc=...)` so 200k rows finish in reasonable time on CPU.
- Build JI-6a: `hi_rules_v1 × {train, eval, flores}`. JI-6b: add trankit and random (random histogram fitted from the default grouper). Upload the whole `boundary_cache/` folder as **one Kaggle Dataset** named `mtp-boundary-cache` (shared with Jainam and Om).
- Test: 20 sentences, `group_id` monotonic, lengths equal, special tokens = -1.

JI-7 • Marathi

Files: mtp/data/grouping/marathi_rules.py (name="mr_rules_v1"), data/gold/mr_gold.jsonl

- Marathi postpositions are mostly **suffixes inside the word** (घरात, घराला, घरासाठी), so fewer separate words attach. Separate attaching words to cover: auxiliaries आहे आहेत होता होती होते होतो असेल असतील नाही; aspect/modal: लागला लागली लागले शकतो शकते शकतात; compound postpositions written separately: च्या साठी, च्या बद्दल, च्या नंतर, च्या आधी, च्या मध्ये (check real text for which are written split vs joined).
- 150 Marathi gold sentences (FLORES `mar_Deva` + IndicCorp `mar_Deva`). Score `mr_rules_v1` vs trankit (mr).
- Build the Marathi caches with the **Marathi model's tokenizer** (`smallstepai/Misa1-1B-instruct-v0.1`) and add them to the same Kaggle Dataset.

JI-8 • Group statistics (motivation numbers for the paper)

File: `scripts/group_stats.py` → `results/grouping/stats_{lang}.json` + 2 plots

- Distribution of group length in words and in tokens (hi vs mr).
- **Key number:** for each lookahead $d \in \{1,2,3,4\}$, the fraction of positions t where token $t+d$ is in the **same group** as token t . This directly tells us how much "within-group" signal each head can exploit, and it's the number the method section opens with.
- Most frequent 30 multi-word groups.

JI-9 • Error analysis by group type

After Om's eval writes per-token dumps (`results/{run}/tokens_{dataset}.jsonl`: `sent_id, text, token_idx, token, head, correct, group_start, group_id`). Group **type** is not in the dump. Re-run your grouper on `text` and attach the type yourself, which is why the grouper should expose `group_types(sentence) → list[str]` alongside `group_words`.

- For R2 vs R5 vs R6: per-head top-1 broken down by group type (aux chain / postposition / compound postposition / light verb / single word) and by position-in-group (first, middle, last).
 - One table + 10 qualitative examples where R5 gets a far-head prediction right that R2 gets wrong, and 5 where R5 is worse.
-

Write-up (Phase E)

- **Data & annotation section:** corpora, grouping rules, guideline summary, gold set size, agreement κ , grouper scores, group statistics.
- **Related work:** MTP (Gloeckle et al. 2024; Medusa; Aynetdinov & Akbik), morphology-aware tokenization (MorphTok), Indic LMs (Al4Bharat, ganga), multi-task loss weighting (Kendall et al. 2018; DWA). Keep a shared `docs/references.bib`.

Om: Evaluation & Infrastructure

How to use this file with Claude: paste the "Context" block below, then `INTERFACES.md`, then the one task you're on (e.g. "Do **OM-4**"). Ask for the module **and** its pytest. Run the test, then open a PR on an `om/<task>` branch.

Context (paste this to Claude)

We are building *Word-Group Guided Multi-Token Prediction for Hindi and Marathi*. A 1B Hindi LM (LingoIITGN/ganga-1b, vocab 30k) is fine-tuned with PEFT LoRA ($r=8$, `q_proj / v_proj`) plus extra "Medusa-style" heads. Head i predicts the token $i+1$ positions ahead, and all heads read the same last hidden state. Training runs on Kaggle (T4/P100 GPUs, 16 GB, sessions that end after a time limit). My job is to make the codebase a clean, reproducible package, make it run on Kaggle with resume, and build the full evaluation suite: per-head perplexity/accuracy, self-speculative decoding speedup, and the tables and figures for the paper.

Existing code (flat in the repo root, to be moved to `legacy/`): `train_ntp_baseline_final.py`, `train_mtp_baseline.py`, `medusa_heads.py`, `word_group_boundaries.py`, `boundary_alignment.py`, `validate_on_real_data.py`, `check_datasets.py`, `test_load_model.py`, `test_lora_training.py`. The training scripts share a `log()` helper that appends to a txt file, a `batchify()` that pads with `tokenizer.pad_token = eos_token`, and a per-head loss that masks padding with -100. Models load with `dtype=torch.bfloat16`, which must become configurable because Kaggle T4/P100 have no native bf16.

Priority: **OM-1** → **OM-2** → **OM-3** come first. Together they are handoffs H1 and H2, which training on Kaggle waits on. Post in the team chat as each one merges.

OM-1 · Package skeleton + shared utilities

Files: `mtp/__init__.py` and all sub-package `__init__.py`, `mtp/config.py`, `mtp/device.py`, `mtp/utils/logging.py`, `mtp/data/corpus.py`, `mtp/data/collate.py`, `requirements.txt`, `pyproject.toml` (so `pip install -e . works`), `README.md`, `legacy/`

- `git mv` every existing root script into `legacy/` unchanged. Keep `.gitignore`, and add `runs/`, `boundary_cache/`, `wandb/`.
- `config.py`: `RunConfig` dataclass with nested dataclasses matching INTERFACES §6. Provide `load_config(path)`, `save_config(cfg, path)`, and CLI overrides like `--set optim.lr=1e-4`.
- `device.py`: `pick_device()`, `pick_dtype(cfg.dtype)`. For `auto: bf16` if `torch.cuda.is_bf16_supported()`, else load weights in `fp32` and train under `torch.autocast(fp16)` with `GradScaler`. Expose a context manager `autocast_ctx(cfg)` and `make_scaler(cfg)` so `train.py` and `evaluate.py` never branch on `dtype` themselves.
- `logging.py`: `MetricLogger(run_dir)` with `.log(step, split, name, value)` → JSONL (INTERFACES §9), plus a pretty console line. Optional W&B mirror if `WANDB_API_KEY` is set.
- `corpus.py`: `load_split(lang, split, n)` implementing the fixed splits in INTERFACES §4 (IndicCorp rows 0-999 = eval, ≥1000 = train, FLORES devtest), with `eval_small` = first 100.
- `collate.py`: `Collator(pad_id)` per INTERFACES §3. It must also work when `group_*` columns are absent (plain NTP runs).
- `requirements.txt`: pin `torch`, `transformers`, `peft`, `datasets`, `pyyaml`, `pytest`. Match the laptop venv versions (`pip freeze` there).

- Tests: config round-trip, collate padding values, `pick_dtype` under mocked `is_bf16_supported`.

OM-2 • Checkpointing + resume

File: `mtp/model/checkpoint.py` (INTERFACES §8)

- `save_run`: PEFT `save_pretrained` for LoRA, `state_dict` for extra heads and the loss-weighting module, optimizer + scaler + RNG states for resume, and `config.yaml` once.
- `load_run`: rebuild base model → apply LoRA → wrap `MTPModel` with `num_heads / head_type` from config → load heads. Use a tiny stand-in for `MTPModel` until Jainam's lands, since only the signature matters.
- `latest_step(run_dir)`, and keep only the last 2 step folders + any folder listed in `keep_steps`.
- Test with a tiny random `LlamaForCausalLM` config (2 layers, hidden 64, vocab 100) so it runs on CPU in seconds.

OM-3 • Kaggle training notebook

File: `notebooks/kaggle_train.ipynb`

- Cell 1: `git clone` the repo at a given branch/commit and `pip install -e .`. Read HF token and (optional) W&B key from **Kaggle Secrets**, never hard-coded.
- Cell 2: attach Kaggle Dataset `mtp-boundary-cache` (from Jai) and set `data.cache_dir`.
- Cell 3: `!python scripts/train.py --config configs/<RUN>.yaml --resume auto`, writing to `/kaggle/working/runs/<run_name>`.
- Resume across sessions: at the end of a session, save `runs/<run_name>` as a Kaggle Dataset version (`kaggle datasets version`) and restore it at the start of the next. Document the 3-click process in `docs/kaggle.md`.
- Print GPU type, dtype chosen, and estimated steps/hour at start.

OM-4 • Evaluation: perplexity + per-head accuracy

Files: `mtp/eval/perplexity.py`, `mtp/eval/head_accuracy.py`

- For each head `i`: mean shifted CE (same masking as `compute_head_loss` in `train_mtp_baseline.py`), perplexity, top-1 and top-5 accuracy.
- **Split by structure** (needs the boundary cache columns): `top1_in_group` = targets whose `group_id` equals the source token's `group_id`, `top1_at_boundary` = the rest. Do the same for loss.
- `--dump_tokens` writes `results/{run}/tokens_{dataset}.jsonl` with `sent_id`, `text`, `token_idx`, `token`, `head`, `correct`, `group_start`, `group_id` (Jai uses this for error analysis).
- Datasets: `indicorp_eval` (1,000), `flores_hi`, later `flores_mr`.
- Test: tiny model, hand-built batch where you know the right answer.

OM-5 • Blind double annotation

- Jai gives you 50 sentence IDs from the Hindi gold set (text only). Annotate them yourself with `scripts/annotate.py` following `docs/annotation_guide.md`, **without** looking at Jai's groups. Save as `data/gold/hi_gold_om50.jsonl`. This gives the inter-annotator agreement number in the paper.

OM-6 • Self-speculative decoding engine

File: `mtp/eval/spec_decode.py` (+ `FixedK` and `ConfidenceCut` in `mtp/eval/draft_policy.py`, contract in INTERFACES §11)

- Loop: run the model on the current sequence → heads 1..k-1 at the last position propose up to k-1 future tokens (greedy argmax) → `policy.num_draft_tokens(...)` decides how many to keep → one forward pass over `context + drafts` → head 0 verifies left to right, accepting while head 0's argmax equals the draft. Always emit at least 1 token (head 0's own prediction at the first mismatch).
- Use the KV cache (`past_key_values`) and roll it back on rejection. Get the no-cache version correct first, then optimise.
- Stats: tokens generated, forward passes, mean accepted length, per-head acceptance rate, tokens/sec, and the same for plain greedy on the same prompts → speedup.
- **Correctness test (must pass):** for 20 prompts, spec-decode output ids == plain greedy output ids, exactly.
- Group Integrity: given a grouper, the fraction of accepted multi-token spans that end on a group boundary.
- Prompts: first 8–16 words of 200 FLORES devtest sentences, `max_new_tokens=64`, batch size 1, timed with `torch.cuda.synchronize()`.

OM-7 • Eval entry point + Kaggle eval notebook

Files: `scripts/evaluate.py`, `notebooks/kaggle_eval.ipynb`

- `python scripts/evaluate.py --run_dir <...> --datasets indicorp_eval flores_hi --spec_decode --policies fixed_k confidence_cut group_aware`
- Writes `results/{run_name}/eval_{dataset}.json` exactly per INTERFACES §10. Commit these JSONs.
- The notebook attaches a run's Kaggle Dataset (checkpoints) + `mtp-boundary-cache` and runs the script. It runs on your own Kaggle GPU quota.

OM-8 • Evaluate every run

- As each run R0–R10 finishes, Jainam posts its run Dataset name. Run **OM-7** on it and push the JSON. Keep `results/STATUS.md` with a checklist of which runs are evaluated on which datasets.
- Sanity-flag anything odd, e.g. head 0 of an MTP run much worse than R0, or speedup < 1.

OM-9 • Tables + figures

File: `scripts/make_tables.py` → `results/tables/*.md` + `*.tex`, `results/figures/*.pdf`

- **Table 1:** main results. Rows R0–R7 (hi), columns: head0 ppl, per-head top-1 (h1–h3), in-group top-1 (h1–h3), mean accepted length, speedup.
- **Table 2:** Marathi (R8–R10). **Table 3:** grouper ablation (R5 vs R6 vs R7). **Table 4:** draft policies (FixedK / ConfidenceCut / GroupAware) on R2 and R5.
- **Figures:** (a) eval loss per head vs step for R2 and R5 (from `metrics.jsonl`); (b) per-head accuracy, in-group vs at-boundary, grouped bars; (c) learned loss weights over training for R5; (d) **lookahead heatmap:** one sentence, each token coloured by the smallest head index that predicted it correctly, with word-group brackets drawn above the tokens. (d) is the paper's headline figure, so make it look good: a Devanagari font (Noto Sans Devanagari), colour-blind-safe palette, and vector PDF.

Write-up (Phase E)

- **Experimental setup:** hardware, dtype handling, hyper-parameters (pull from configs automatically), eval sets, decoding setup.
- **Evaluation section:** metric definitions (per-head accuracy, in-group split, acceptance length, Group Integrity), all tables and figures, appendix with full per-run numbers.

Interface Contracts

Project: Word-Group Guided Multi-Token Prediction with Adaptive Loss Weighting for Hindi and Marathi

This file is the single source of truth for how the pieces connect. Every task file points here. If you need to change a contract, change it here first and tell the other two people. Code that matches these signatures can be written by any of us independently and will plug together.

0. Ground truth: what already exists in the repo

File	What it does	Status
<code>test_load_model.py</code>	Loads <code>LingoIITGN/ganga-1b</code> , generates Hindi	Done
<code>test_lora_training.py</code>	PEFT/LoRA wrap + 3 dummy steps	Done
<code>check_datasets.py</code>	Loads <code>ai4bharat/IndicCorpV2</code> , config <code>indiccorp_v2</code> , split <code>hin_Deva</code>	Done
<code>train_ntp_baseline_final.py</code>	NTP baseline: LoRA <code>r=8</code> on <code>q_proj, v_proj</code> , lr <code>2e-4</code> , batch 8, max_len 128, 100 held-out sentences, grad clip 1.0	Done. Held-out loss 3.05
<code>medusa_heads.py</code>	<code>MedusaWrapper(base_model, num_extra_heads)</code> : extra heads are plain <code>nn.Linear(hidden, vocab)</code> on the last hidden state	Done (k=2 tested)
<code>train_mtp_baseline.py</code>	Token-level MTP, k=2, same LoRA setup, summed per-head CE	Done. 12,500 batches: head0 3.09 , head1 8.10 → 5.84
<code>word_group_boundaries.py</code>	<code>find_word_groups(sentence) → list[list[str]]</code> , rule-based (aux + postposition lists)	v0. The on-disk file still has the short lists (the expanded lists were never saved)
<code>boundary_alignment.py</code>	<code>get_aligned_boundary_labels(sentence)</code> returns token-level <code>1 = group start</code> labels via <code>offset_mapping</code>	Done, verified by eye
<code>validate_on_real_data.py</code>	Coverage stats on real corpus text. Last result: 25% words matched, 1.33 words/group	Done

These all live flat in the repo root. Step 1 of the plan moves them into the package below (old copies go to `legacy/`).

1. Target repo layout

```

mtp-indic-research/
├── mtp/                                # importable package: import mtp
│   ├── __init__.py
│   ├── config.py                      # RunConfig dataclass + YAML load/save      [Om]
│   ├── device.py                     # dtype/autocast selection (bf16 vs fp16 vs fp32) [Om]
│   └── data/
│       ├── corpus.py                 # streaming load, fixed held-out split, batching [Om]
│       ├── collate.py                # pads input_ids / attention_mask / group_id [Om]
│       ├── boundary_cache.py         # build + load pre-labelled datasets      [Jai]
│       └── grouping/
│           ├── base.py               # Grouper protocol + registry              [Jai]
│           ├── hindi_rules.py        # rule grouper, Hindi                      [Jai]
│           ├── marathi_rules.py      # rule grouper, Marathi                    [Jai]
│           ├── trankit_grouper.py    # dependency-parse grouper (hi + mr)      [Jai]
│           ├── random_grouper.py     # control: random boundaries, same length dist. [Jai]
│           └── align.py              # word groups → token labels (from boundary_alignment.py)
├── [Jai]
│   ├── model/
│   │   ├── heads.py                 # MTPModel, head variants                  [Jainam]
│   │   └── checkpoint.py            # save_run / load_run                      [Om]
│   ├── losses/
│   │   ├── mtp_ce.py               # per-head shifted CE (from train_mtp_baseline.py) [Jainam]
│   │   ├── structural.py           # word-group structural loss                [Jainam]
│   │   ├── contrastive.py          # group contrastive loss                   [Jainam]
│   │   └── weighting.py            # fixed / uncertainty / DWA weighting      [Jainam]
│   ├── eval/
│   │   ├── perplexity.py           # per-head loss + ppl                      [Om]
│   │   ├── head_accuracy.py        # top-1/top-5 per head, split in-group vs boundary [Om]
│   │   ├── spec_decode.py          # self-speculative decoding engine + speed metrics [Om]
│   │   ├── draft_policy.py         # pluggable draft-length policies          [Jainam]
│   │   └── group_metrics.py        # grouper P/R/F1 vs gold, group stats      [Jai]
│   └── utils/
│       └── logging.py              # JSONL metric logger                      [Om]
├── scripts/
│   ├── train.py                   # single training entry point              [Jainam]
│   ├── evaluate.py                # single eval entry point                  [Om]
│   ├── build_boundary_cache.py     # CLI around boundary_cache               [Jai]
│   ├── score_grouper.py           # grouper vs gold set                     [Jai]
│   └── make_tables.py             # results/*.json → tables + plots         [Om]
├── configs/                       # one YAML per run (see §6)               [Jainam]
├── owns run configs]
├── notebooks/
│   ├── kaggle_train.ipynb         [Om]
│   └── kaggle_eval.ipynb          [Om]
├── data/gold/                     # hand-annotated gold groups              [Jai]
├── docs/                          # annotation guide, plan, report drafts
├── results/                       # eval JSON + tables + figures (committed)
├── tests/                         # pytest, CPU-only, tiny inputs
├── legacy/                        # the original root-level scripts, untouched
├── requirements.txt
└── README.md

```

Rule: **nothing under mtp/ may download a model or dataset at import time.** Loading happens inside functions.

2. Grouping contract (Jai builds, everyone consumes)

```
# mtp/data/grouping/base.py
from typing import Protocol

class Grouper(Protocol):
    name: str          # e.g. "hi_rules_v1", "trankit", "random"
    lang: str          # "hi" or "mr"
    def group_words(self, sentence: str) → list[list[str]]:
        """Whitespace words of `sentence`, partitioned into contiguous groups.
        Concatenating all groups in order MUST give exactly sentence.split()."""
    def group_types(self, sentence: str) → list[str]:
        """One type per group, same order as group_words():
        'single' | 'aux_chain' | 'postposition' | 'compound_postposition' | 'light_verb' |
        'other'"""

    def get_grouper(name: str, lang: str, **kwargs) → Grouper: ...
```

```
# mtp/data/grouping/align.py
def label_tokens(sentence: str, tokenizer, grouper: Grouper, max_length: int = 128) → dict:
    """Tokenize the FULL sentence once (same call as training) and return:
        input_ids:      list[int]
        attention_mask: list[int]
        group_start:    list[int] # 1 if token is first token of a word-group, else 0
        group_id:       list[int] # 0,0,0,1,1,2,... index of the group each token belongs to
                                # -1 for special tokens (BOS/EOS) and padding
    All four lists have identical length."""
```

Rules:

- Uses `offset_mapping`, same as `boundary_alignment.py` (SentencePiece leading-space handling included).
- `group_id` is monotonic non-decreasing over real tokens.
- Truncation at `max_length` may cut a group. That is fine; the cut group just ends early.

3. Boundary cache contract (Jai builds, Jainam trains on it, Om evaluates on it)

A HuggingFace `datasets.Dataset` saved with `save_to_disk`, uploaded as a Kaggle Dataset.

column	type	notes
<code>text</code>	<code>str</code>	raw sentence
<code>input_ids</code>	<code>list[int]</code>	unpadded
<code>attention_mask</code>	<code>list[int]</code>	unpadded
<code>group_start</code>	<code>list[int]</code>	\$2
<code>group_id</code>	<code>list[int]</code>	\$2

Folder naming: `boundary_cache/{lang}_{grouper}_{split}/` e.g. `hi_hi_rules_v1_train/`, `hi_trankit_eval/`, `mr_mr_rules_v1_train/`. Splits: `train` (N configurable, default 200k), `eval` (the **fixed** held-out set, see §4), `flres` (FLORES-200 devtest). A `meta.json` next to each folder: grouper name, tokenizer name, `max_length`, n rows, avg group length (words + tokens), % tokens that are group starts.

Collate (`mtp/data/collate.py`, Om): right-pad `input_ids` with pad id, `attention_mask` with 0, `group_start` with 0, `group_id` with -1. Returns tensors.

4. Fixed evaluation sets

So numbers from different people are comparable:

- **IndicCorp held-out:** first 1,000 rows of the `hin_Deva` stream (and `mar_Deva` for Marathi). Training draws from row 1,000 onward. (The current scripts use the first 100. Keep that subset as `eval_small` for quick checks.)
- **FLORES-200 devtest:** `hin_Deva`, `mar_Deva`.
- **Gold grouping set:** `data/gold/hi_gold.jsonl`, `data/gold/mr_gold.jsonl` (§7).

5. Model contract (Jainam builds)

```
# mtp/model/heads.py
@dataclass
class MTPOutput:
    logits: list[torch.Tensor] # length k; logits[i] is [B, T, V]; head i predicts token t+i+1
    hidden: torch.Tensor      # [B, T, H] last hidden state (for contrastive loss)
    aux: dict                  # optional extras, e.g. boundary-prediction logits

class MTPModel(nn.Module):
    def __init__(self, base_model, num_heads: int, head_type: str = "linear", **kw): ...
    def forward(self, input_ids, attention_mask=None) → MTPOutput: ...
```

- `num_heads` counts **all** heads including head 0 (the original LM head). Current `medusa_heads.py` uses `num_extra_heads`; `num_heads = num_extra_heads + 1`.
- `head_type`: "linear" (current), "resblock" (Medusa-1 style residual block, init so it starts as identity + the LM head).

6. Run config contract (Om builds loader, Jainam writes configs)

```
run_name: hi_mtp_k4_struct_adaptive
lang: hi
model_name: LingoIITGN/ganga-1b
num_heads: 4
head_type: resblock
lora: {r: 8, alpha: 16, dropout: 0.05, targets: [q_proj, v_proj]}
optim: {lr: 2.0e-4, batch_size: 8, grad_clip: 1.0, max_steps: 12500}
data: {cache_dir: /kaggle/input/mtp-boundary-cache, grouper: hi_rules_v1, max_length: 128}
losses:
    structural: {enabled: true, variant: S2, weight: 0.1}
    contrastive: {enabled: false, weight: 0.05, temperature: 0.1}
weighting: {scheme: uncertainty} # fixed | uncertainty | dwa
eval_every: 250
save_every: 500
seed: 42
dtype: auto # auto | bf16 | fp16 | fp32
```

7. Gold annotation format (Jai)

`data/gold/hi_gold.jsonl`, one sentence per line:

```
{
  "id": "hi_0001",
  "source": "flores_devtest",
  "text": "मैं कल बाजार जा रहा था।",
  "groups": [[0],[1],[2],[3,4,5]],
  "annotator": "jai",
  "notes": ""
}
```

`groups` are lists of word indices into `text.split()`.

8. Checkpoint contract (Om builds, Jainam calls)

```
# mtp/model/checkpoint.py
def save_run(run_dir: str, model: MTPModel, cfg: RunConfig, step: int, optimizer=None,
weighting=None) → None
def load_run(run_dir: str, device: str = "auto") → tuple[MTPModel, "Tokenizer", RunConfig]
def latest_step(run_dir: str) → int | None # for resuming after a Kaggle session ends
```

Layout:

```
runs/{run_name}/
  config.yaml
  step_{N}/lora/          # PEFT save_pretrained
  step_{N}/heads.pt       # extra heads state_dict
  step_{N}/weighting.pt   # learned loss weights, if any
  step_{N}/optim.pt       # optional, for resume
  metrics.jsonl
```

9. Metrics log contract (Om builds, everyone writes)

metrics.jsonl, one JSON object per line:

```
{ "step": 500, "split": "train", "name": "loss/head1", "value": 6.12 }
{ "step": 500, "split": "eval", "name": "loss/head0", "value": 3.07 }
{ "step": 500, "split": "train", "name": "weight/structural", "value": 0.083 }
```

Name prefixes: loss/, acc/, weight/, lr, time/.

10. Eval output contract (Om builds)

results/{run_name}/eval_{dataset}.json:

```
{
  "run_name": "...", "dataset": "flores_hi", "step": 12500,
  "per_head": [{ "head": 0, "loss": 3.05, "ppl": 21.1, "top1": 0.41, "top5": 0.63,
                  "top1_in_group": 0.55, "top1_at_boundary": 0.33 }],
  "spec_decode": { "policy": "fixed_k", "mean_accepted_len": 1.42, "accept_rate_per_head": [0.42],
                   "tokens_per_sec": 31.5, "greedy_tokens_per_sec": 24.0, "speedup": 1.31,
                   "outputs_match_greedy": true },
  "group_integrity": 0.71
}
```

11. Draft policy contract (Om builds engine, Jainam writes policies)

```
# mtp/eval/draft_policy.py
class DraftPolicy(Protocol):
    name: str
    def num_draft_tokens(self, head_logits: list[torch.Tensor], context_ids: torch.Tensor,
step_state: dict) → int:
        """Return how many of the k-1 drafted tokens to propose this step (0..k-1)."""

class FixedK:          # always propose all k-1          (baseline, Om ships this one)
class ConfidenceCut:   # stop at first head whose max prob < tau
class GroupAware:      # stop at predicted group boundary (Jainam)
```

spec_decode.generate(model, tokenizer, prompt_ids, max_new_tokens, policy) → (output_ids, stats).

Greedy verification: final output **must** be identical to plain greedy decoding with head 0. That is the

correctness test.